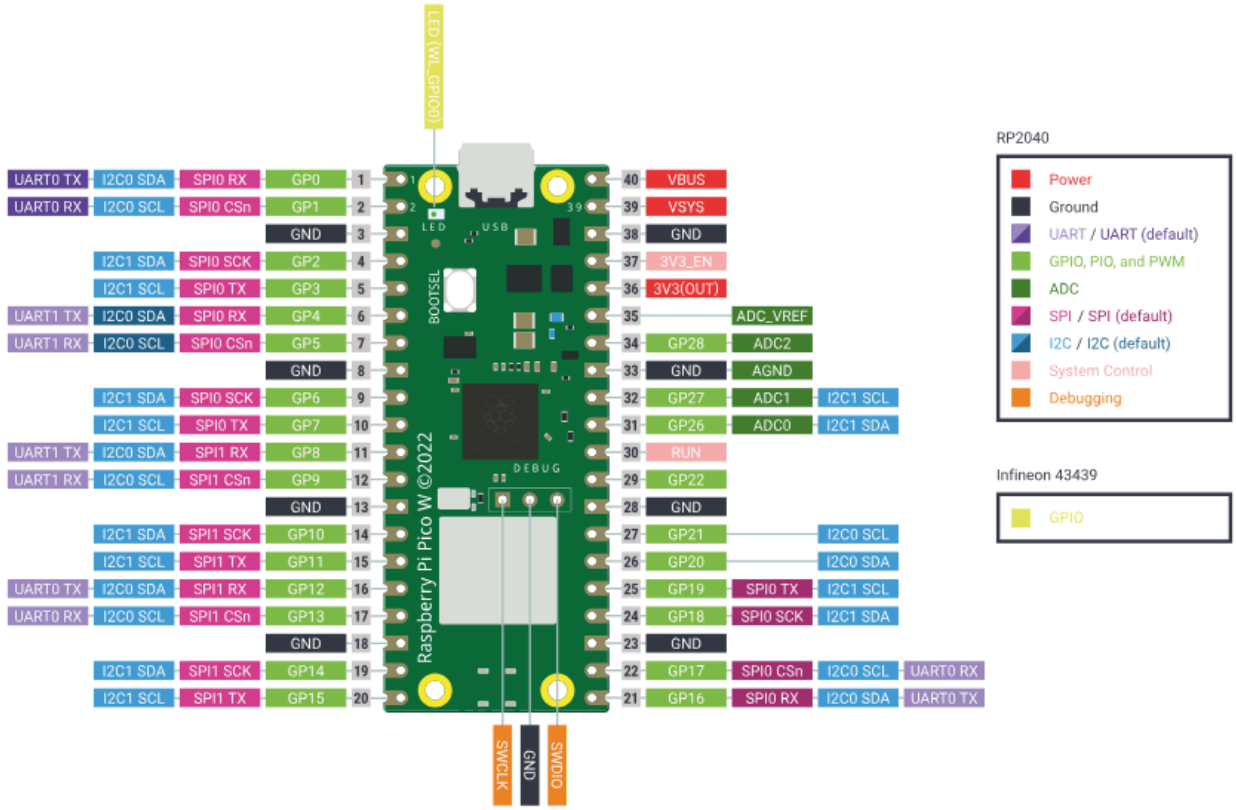
Wprowadzenie. Płytka Raspberry Pi Pico W, w porównaniu z Raspberry Pi Pico, jest wyposażona w układ Infineon CYW43439, który pozwala na komunikację w paśmie 2.4GHz (802.11n). Podstawowe parametry techniczne i funkcjonalne są następujące:

- wsparcie dla WPA3;
- wbudowana antena, na licencji firmy ABRACON (poprzednio ProAnt)
- możliwość uruchomienia punktu dostępowego dla max. 4 klientów;
- interfejs Bluetooth 5.2.

Pozostałe parametry takie, jak GPIO, interfejsy przewodowe, rdzenie obliczeniowe, pamięć danych, pamięć programu są identyczne jak w przypadku płytki Raspberry Pi Pico.



Układ do komunikacji bezprzewodowej jest podłączony do mikrokontrolera RP2040 za pomocą interfejsu SPI.

UWAGA. Ze względu na ograniczenia w ilości pinów, niektóre piny, które są wykorzystane do interfejsu bezprzewodowego mają ograniczoną funkcjonalność. CLK jest współdzielony z VSYS, co oznacza, że odczyt VSYS jest możliwy w przypadku braku przesyłu danych z wykorzystaniem SPI. Linia DIN/DOUT układ Infineon CYW43439 jest podłączona do układu RP2040 poprzez GPIO24. Oznacza to, że przerwania związane z tym GPIO mogą nie działać poprawnie w przypadku transmisji z wykorzystaniem interfejsu SPI.

Zad 1. Skonfiguruj płytkę Raspberry Pi Pico W do pracy ze środowiskiem Thonny IDE. Niezbędne informacje znajdują się na stronie:

`projects.raspberrypi.org/en/projects/get-started-pico-w/1`

Wszystkie kody są dostępne w repozytorium GitHub, w pliku - `ex05_wifi.zip`.

Zad 2. Podłącz płytkę Raspberry Pi Pico W do bezprzewodowej sieci `Szkolenia`.

```
import time
from network import WLAN, STA_IF
import gc

ssid = "Szkolenia"
password = "Doktorzec1"

gc.collect()

wlan = WLAN(STA_IF)
wlan.active(True)
wlan.connect(ssid,password)

max_wait = 10
```

```
    while max_wait > 0:
        if wlan.status() < 0 or wlan.status() >= 3:
            break
        max_wait -= 1
        print("Connecting to ",ssid)
        time.sleep(1)

    if wlan.status() != 3:
        raise RuntimeError("Network connection failed")
    else:
        print("Connected")
        status = wlan.ifconfig()
        print("IP adres " + status[0])
    wlan.disconnect()
```

Kody, określające status połączenia, są opisane na stronie 16, wdokumentacji *Connecting to the Internet with Raspberry Pi Pico W*:

www.raspberrypi.com/documentation/microcontrollers/raspberry-pi-pico.html

Zad 3. Funkcja `wifi_connect()`.

```
from time import sleep
from network import WLAN,STA_IF
from machine import reset
import gc

gc.collect()
ssid = "ASUS"
password = "Doktorzec1"

def wifi_connect():
    wlan = WLAN(STA_IF)
    wlan.active(True)
    wlan.connect(ssid,password)
    while wlan.isconnected() == False:
        print("Connection to " + ssid)
        sleep(1)
    print(wlan.ifconfig())
    print(f'Connected to {ssid} network, IP {wlan.ifconfig()[0]}')
```

```
try:
    wifi_connect()
except KeyboardInterrupt:
    reset()
```

Zad 4. Asynchroniczna implementacja serwera. Podłącz LED do GPIO 15.

```
from time import sleep
from network import WLAN,STA_IF
from machine import reset,Pin
import uasyncio as asyncio
import gc

gc.collect()

def web_page(led_state):
    html = """
    <html>
    <head>
        <meta name="viewport" content="width=device-width,
                     initial-scale=1">
        <link rel="stylesheet"
           href="https://use.fontawesome.com/releases/v5.7.2/css/all.
           css">
        <style>
            html {
                font-family: Arial;
                display: inline-block;
                margin: 0px auto;
                text-align: center;
            }

            .button {
                background-color: #ce1b0e;
                border: none;
                color: white;
                padding: 16px 40px;
                text-align: center;
                text-decoration: none;
                display: inline-block;
```

```
                font-size: 16px;
                margin: 4px 2px;
                cursor: pointer;
            }

            .button1 {
                background-color: #000000;
            }
        </style>
    </head>
    <body>
        <h2>RPi Pico W based web server</h2>
        <p>LED state: <strong>""" + led_state + """</strong></p>
        <p>
            <i class="fas fa-lightbulb fa-3x" style="color:#c81919;">
                </i>
            <a href=\"led_on\"><button class="button">LED ON</button>
                </a>
        </p>
        <p>
            <i class="far fa-lightbulb fa-3x" style="color:#000000;">
                </i>
            <a href=\"led_off\"><button class="button button1">LED
                OFF</button></a>
        </p>
    </body>
    </html>"""
    return html


ssid = "Laboratorium-IoT"
password = "enter the password here"

led = Pin(15,Pin.OUT)
led_state = "LED is off"
onboard = Pin("LED",Pin.OUT,value = 0)

def wifi_connect():
    wlan = WLAN(STA_IF)
    wlan.active(True)
    wlan.connect(ssid,password)
    while wlan.isconnected() == False:
```

```
        print("Connection to " + ssid)
        sleep(1)
    print(wlan.ifconfig())
    print(f'Connected to {ssid} network, IP {wlan.ifconfig()[0]}')

async def handle_client(reader,writer):
    print("Client connected")
    request_line = await reader.readline()
    print("Request:",request_line)

    #skip header
    while await reader.readline() != b"\r\n":
        pass

    request = str(request_line)
    led_on = request.find('led_on')
    led_off = request.find('led_off')
    print('led on = ' + str(led_on))
    print('led off = ' + str(led_off))

    if led_on == 7:
        print("led on")
        led.value(1)
        led_state = "LED is ON"

    if led_off == 7:
        print("led off")
        led.value(0)
        led_state = "LED is OFF"

    response = web_page(led_state)
    writer.write('HTTP/1.0 200 OK\r\nContent-type:
text/html\r\n\r\n')
    writer.write(response)

    await writer.drain()
    await writer.wait_closed()
    print("Client disconnected")

async def main():
    try:
        wifi_connect()
```

```
    except KeyboardInterrupt:
        reset()

    print("Starting webserver")
    asyncio.create_task(asyncio.start_server(handle_client,
"0.0.0.0", 80))
    while True:
        onboard.on()
        #print("server is still alive")
        await asyncio.sleep(0.25)
        onboard.off()
        await asyncio.sleep(5)

try:
    asyncio.run(main())
finally:
    asyncio.new_event_loop()
```

Zad 5. Zaimpementuj kod.

```
from time import sleep
from network import WLAN,STA_IF
from machine import reset
import urequests as rq
import json

ssid = "Szkolenia"
password = "enter the password here"

def wifi_connect():
    wlan = WLAN(STA_IF)
    wlan.active(True)
    wlan.connect(ssid,password)
    while wlan.isconnected() == False:
        print("Connection to " + ssid)
        sleep(1)
```

```
    print(wlan.ifconfig())
    print(f'Connected to {ssid} network, IP {wlan.ifconfig()[0]}')

try:
    wifi_connect()
except KeyboardInterrupt:
    reset()

print("\n\n2. Querying the current GMT+0 time:")
date_time =
     rq.get("http://https://time.now/developer/api/timezone/Europe/P
     aris")
print(date_time.json())
today = 'Today is ' + date_time.json().get('datetime')
print(today)
```

Zad 6.(**0.5 pkt.**) Podłącz wyświetlacz *Pico-LCD-x.xx* do płytki Raspberry Pi Pico W.
Na wyświetlaczu powinien być wyświetlany aktualny czas z rozdzielczościa do 1s oraz data.

Zad 7.(**1 pkt.**) Podłącz wyświetlacz *Pico-LCD-x.xx* do płytki Raspberry Pi Pico W.
Napisz skrypt, który pobiera temperaturę z serwisu *OpenWeather* (openweathermap.org) i wyświetla ją na wyświetlaczu w stopniach Celsjusza i w Kelwinach.

Zad 8.(**1.5 pkt.**) Pobieraj, w czasie rzeczywistym co najmniej 3 parametry opisujące pogodę, z z serwisu *OpenWeather* (openweathermap.org). Wyświetl wynika na stronie www oraz wyświetlaczu *Pico-LCD-x.xx*. Dodaj ikonę lub inny element graficzny dla każdego parametru opisującego

pogodę. Co najmniej 1 parametr powinien być wyświetlany na wyświetlaczu lub wskaźniku analogowym. Na wyświetlaczu wystarczy wyświetlać wyniki w formie numerycznej.

Dla zainteresowanych:

1. Raspberry Pi Pico W vs. Pico. Jakie są różnice?
   core-electronics.com.au/guides/raspberry-pi-pico-w-vs-pico-whats-the-difference/

2. Strona projektu MicroPython:
   micropython.org/download/rp2-pico/

3. Konfiguracja Raspberry Pi Pico W:
   projects.raspberrypi.org/en/projects/get-started-pico-w/

4. Dokumentacja do Raspberry Pi Pico oraz Pico W:
   www.raspberrypi.com/documentation/microcontrollers/raspberry-pi-pico.html

5. Dokumentacja modułów Pico-LCD:
   www.waveshare.com/wiki/Pico-LCD-0.96